601.433/633 Introduction to Algorithms **Fall 2026**
Homework #7 **Due:** November 5, 2026, 11:59pm

Remember: this is an oral presentation homework! You are required to work in groups of three, and may collaborate only with other students in your group. We encourage you to write up solutions, but your grade will be based entirely on your presentation. At your presentation, the grader will randomly assign each student to present a problem. You should be able to create a group and sign up for a slot using Canvas. The deadline for signing up for a presentation slot is 5pm Wed Nov 4.

You may use the internet or LLMs to look up formulas, definitions, concepts, etc., but may not simply look up the answers online or ask an LLM.

1 Faster Shortest Paths

Let $G = (V, E)$ be a directed graph with lengths $\ell : E \to \mathbb{R}$ with no negative-length cycles. Let $v \in V$, and let α denote the maximum, over all $u \in V$, of the number of edges on a shortest path from v to u (where the shortest-path is defined with respect to the weights). Given G, w, and v (but not α), give an algorithm that computes shortest path distances from v to all other nodes in $O(m\alpha)$ time. You may assume that $m = \Omega(n)$. Prove correctness and running time.

Algorithm Description:

We use Bellman-Ford. This algorithm finds the shortest distance between a starting node v in graph G and all other nodes in G. Bellman-Ford's correctness proof shows that after i relaxations, $OPT(u, i)$ would find the shortest path from v to u with at most i edges, as per **Theorem 15.3.1**. Therefore, if we perform α rounds of relaxations, we would find the shortest path between v and u with at most α edges. Since α is the maxium number of edges needed in the path from v to any reachable u, this means that all possible paths from v to any reachable node u would be found.

However, as α is not given, we must find when to stop the algorithm. However, we can set up a variable track changes in distance estimates. After each round, this variable is set to False. Once the shortest path between v and every u has been found, on the next iteration, there will be no updates to the distance estimate. Thus, the algorithm would return. Since it takes α passes to find all shortest paths, there would be no updates to the distance estimate tracker on the $(\alpha+1)$th iteration. Therefore, there would be $\alpha + 1$ iterations of the loop.

Correctness Proof:

Bellman-Ford correctness proof + def of alpha

Runtime Proof:

As per **Theorem 15.2.3**, we know each iteration of the outer loop of the algorithm takes $O(m)$ time. Since the outer loop is repeated $\alpha + 1$ times, this means the total runtime is $O(m) \cdot (\alpha + 1)$, or $O(m\alpha)$.

2 Multi-Set Shortest Paths

Let $G = (V, E)$ be a directed graph with weighted edges; edge weights can be positive, negative, or zero. There are no negative-weight cycles. Suppose the vertices of G are partitioned into k disjoint

subsets $V_1, V_2, \ldots, V_k$; that is, every vertex of G belongs to exactly one subset V_i. For each $i, j \in [k]$, let $\delta(i, j)$ denote the minimum shortest-path distance between vertices in V_i and vertices in V_j:

$$\delta(i, j) = \min \{d(v_i, v_j) \mid v_i \in V_i \text{ and } v_j \in V_j\}$$

Design an algorithm to compute $\delta(i, j)$ for all $i, j \in [k]$ that runs in time $O(mn + kn \log n)$. You may assume that $m = \Omega(n)$. Prove correctness and running time.

Algorithm Description:

We use a modified version of Johnson's algorithm to solve this problem. The first part, where we run Bellman-Ford is the same. We find vertex potentials and reweight all edges to eliminate negative weights. This is the same as Bellman Ford with additional $O(1)$ computations per loop iteration, so it takes $O(mn)time$.

From here, we can run multi source Dijkstra, but we only need to find the distance between two subsets, not the distance between every node. Therefore, we run Dijkstra's once from each subset instead.

We create a source vertex s_i for each set V_i and we set its distance from each vertex in V_i to $C_i - h(u)$, where C_i is the maximum potential in V_i, and $h(u)$ is the potential of any vertex u in V_i. This cancells out the reweighting of Johnson's algorithm, and introduces C_i as a means of normalization to keep all distances nonnegative.

From here, we run Dijkstra's on each source s_i, and we keep track of the minimum distance $D[k]$ from each s_i to each set. We run this for each of the k sources, and yield the minimum distance from each set to each other set.

Correctness Proof:

As per lecture notes, Johnson works. As per lecture notes, Dijkstra works when all edges positive, which it is after Johnson reweighting. The calculated distance in Dijkstra's $d'(u, v)$ is:

$d'(u, v) = d(u, v) + h(u) - h(v)$

Thus, for each set V_i, we want the algorithm to return:

$D_i(v) = \min_{u \in V_i}(d'(u, v)) = \min_{u \in V_i}(d(u, v) + h(u) - h(v))$

Since the starting distance between the source and u is $C_i - h(u)$, we must add that to this equation to correctly find the minimum:

$D_i(v) = \min_{u \in V_i}(C_i - h(u) + d(u, v) + h(u) - h(v)) = \min_{u \in V_i}(C_i + d(u, v) - h(v))$

Since C_i and $h(v)$ are constants in this context, we can rearrage:

$\min_{u \in V_i}(d(u, v)) = D_i(v) - C_i + h_v$

As $\delta(u, v) = \min_{u \in V_i}(d(u, v))$, we can see that this equation correctly finds the distance. Therefore, the changed source weights allow for accurate calculation of any $\delta(u, v)$.

Runtime Proof:

Bellman-Ford is $O(mn)$. Dijkstra is $O(n \log n)$. We run it k times. This means $O(mn) + k \cdot O(n \log n) = O(mn + kn \log n)$.

3 Single-Source Shortest Paths

Let $G = (V, E)$ be a connected, undirected graph with positive edge lengths $\ell : E \to \mathbb{R}^+$. This graph represents a road network. You are trying to get from $s \in V$ to $t \in V$. Unfortunately, you

are a coffee addict: getting caffeine often enough is the most important thing to you. In fact, there is some number $L \in \mathbb{R}^+$ so that if you go more that L distance without getting coffee, you will get a caffeine migraine and crash your car. So you simply *cannot* go more than L distance without stopping for coffee. Fortunately, there is a coffee shop at every vertex $v \in V$ (including s and t) where you can get coffee. However, you are also a coffee snob, and you *hate* Starbucks. And there is a special subset of nodes $S \subseteq V$ where the only coffee shop on that node is a Starbucks.

So you have the following problem: given a subset $S \subseteq V$ of vertices where the only coffee shop is a Starbucks, find the path from s to t which uses only edges of length at most L, and has the *fewest* vertices in S. You may assume that $s, t \notin S$. Design an $O(m + n \log n)$-time algorithm for this problem, and prove running time and correctness.

Algorithm Description:

We use a modified version of Dijkstra's algorithm. We first perform an $O(m)$ pass of all of the edges in G, and delete edges with lengths greater than L, as we cannot travel that distance. As G is undirected, we consider each edge $v \leftrightarrow u$ to be 2 edges: $v \leftarrow u$ and $v \rightarrow u$. Additionally, on this pass, we reassign all of the valid edge weights $v \rightarrow u$ based on the following:

(1) $u \in S$: $w(v \rightarrow u) = 1$
(2) $u \notin S$: $w(v \rightarrow u) = 0$

From here, we run Dijkstra's algorithm on the modified graph. Since the problem asks to minimize the number of vertices in S, we can consider visiting vertices not in S to be "free", as is reflected in our recomputed edge weights. Thus, running Dijkstra's on this will find the path that minimizes the number of visits to vertices in S.

Correctness Proof:

Dijkstra's is correct as per **Theorem 15.4.1**. Dijkstra's will find the lowest-cost path from s to t. Visiting vertices not in S is free, but those in S cost 1. Therefore, the lowest0cost path is the one with the fewest Starbucks.

Runtime Proof:

Initial pass is $O(m)$. Dijkstra is $O(m + n \log n)$. $O(m) + O(m + n \log n) = O(m + n \log n)$